

Tuples, Dictionary, and Sets in Python



Agenda

☒ ~~What is Python? Why Python? How to use it?~~

☒ ~~Python basics~~

☒ ~~Lists~~

☒ **Tuples**

☒ **Dictionary and sets**

☐ Functions

☐ Classes

☐ Try-Catch exception

Agenda

☒ ~~What is Python? Why Python? How to use it?~~

☒ ~~Python basics~~

☒ ~~Lists~~

☒ **Tuples**

☒ **Dictionary and sets**

☐ Functions

☐ Classes

☐ Try-Catch exception

About Tuples in Python

What is Tuple?

- A tuple is an **ordered collection** of items and is **immutable**.
- Like lists, tuples can hold different data types and allow duplicate members.
- They are defined by placing the items inside parentheses ().

Why tuples?

- **Data integrity:** Because tuples are immutable, they protect data from accidental modification. This is especially useful for constants or configuration settings.
- **Performance:** Tuples are generally faster than lists. If you are just iterating through a collection and not modifying it, a tuple is more efficient.

How to use them?

You create a tuple by placing items inside **parentheses ()**, separated by commas.

About Tuples in Python

Summary:

1. Tuple **size is fixed**, so no `extend()` or `append()`
2. You can't **remove or insert elements**, so no `insert()`, `remove()`, or `pop()` methods
3. Tuples are **ordered**, which means you can use index to search for specific element just like lists
4. Tuples are **computationally faster** than lists

It makes sense to use for write-protected data.

Tuple declaration

You create a tuple using rounded brackets, in which items are separated by commas.

```
x = (1, 2, 3, 4, 5)
print(x)
print(type(x))
```

Output:

```
(1, 2, 3, 4, 5)
<class 'tuple'>
```



```
y = 6, 7, 8, 9, 10 #Without bracket also can create tuple.
print(type(y))
```

Output:

```
<class 'tuple'>
```



Assessing items in Tuple is same as list.

```
x[3]
```

Output: 4



Tuple Manipulation

Unlike lists however, any manipulation of tuple elements is not allowed.

```
x = (1, 2, 3, 4, 5)
x[3] = 9
```



Output:

```
TypeError: 'tuple' object does not support item assignment
```

Tuple Packing and Unpacking

Packing is when you create a tuple. When you write `some_tup = ("text", 1, False)`, you are "**packing**" those three values into a single variable named `some_tup`.

Unpacking is the opposite. It's when you take the items from a tuple and **assign each one to its own variable** in a single line.

```
some_tup = ("text", 1, False) #Packing
x, y, z = some_tup #Unpacking
print(x)
print(y)
print(z)
```

Output:

```
text
1
False
```



Tuple Packing and Unpacking

While unpacking what if we don't want some values? It's a good practice to dump the unwanted value to underscore (_) variable.

```
person = ("Jack", 72, "Male") #Packing
name, _, gender = person #Unpacking with underscore
print (name, gender, sep=": ")
```

Output:

Jack: Male



```
person = ("Jack", 72, "Male") #Packing
name, gender = person #Unpacking with fewer variable
print (name, gender, sep=": ")
```

Output:

ValueError: too many values to unpack (expected 2)



QUIZ Time

Time Duration: 3 Mins

```
people = (("Donald", 78, "Male"), ("Joe", 82, "Male"), ("Hillary", 60, "Female"))
```

From the above tuple, print out only the name of each person

QUIZ Time

```
people = (("Donald", 78, "Male"), ("Joe", 82, "Male"), ("Hillary", 60, "Female"))
```

From the above tuple, print out only the name of each person

#Solution 1: Loop and access item

```
people = (("Donald", 78, "Male"), ("Joe", 82, "Male"), ("Hillary", 60, "Female"))
for person in people:
    name = person[0]
    print(name)
```

#Solution 2: Loop and unpacking

```
for person in people:
    name, _, _ = person
    print(name)
```

#Solution 3: Use unpacking

```
names = [name for person in people for name, _, _ in [person]]
print(names)
```

OUTPUT:

```
Donald
Joe
Hillary
```

Agenda

☒ ~~What is Python? Why Python? How to use it?~~

☒ ~~Python basics~~

☒ ~~Lists~~

☐ **Tuples**

☐ **Dictionary and sets**

☐ Functions

☐ Classes

☐ Try-Catch exception

About Dictionary in Python

What is Dictionary?

- A dictionary in Python is a collection of items that are stored in **key-value pairs**.
- Unlike a list or a tuple, which are ordered by a numerical index, a dictionary is an **unordered collection** where each item is **accessed by a unique key**.

Why Dictionary?

- **Fast Lookups:** Finding a value by its key is very fast in a dictionary, no matter how large it is.
- **Clearer Data:** Using descriptive keys like "name" or "age" is much clearer than remembering that an item is at index 0 or 1.

How to use them?

Dictionaries are created using curly brackets { }, where each element is composed of key and value {key1 : value1, key2 : value2, ...}

Example → `my_dictionary = {"name": "Alice", "age": 30, "city": "New York"}`

Dictionary declaration

You create a dictionary using curly brackets {}.

```
person_dict = {  
    "Name": "Jane",  
    "Age": 30,  
    "Gender": "Female",  
    "Occupation": "Student"  
}  
print(person_dict)  
print(type(person_dict))
```



Output:

```
{'Name': 'Jane', 'Age': 30, 'Gender': 'Female', 'Occupation':  
'Student'}  
<class 'dict'>
```

```
person_dict["Gender"]    ->    Output: "Female"
```

Dictionary Data Update

```
person_dict = {'Name': 'Jane', 'Age': 30, 'Gender': 'Female',  
               'Occupation': 'Student'}
```

```
#change age to 31
```

```
person_dict["Age"] = 31  
print(person_dict)
```

**Output:**

```
{'Name': 'Jane', 'Age': 31, 'Gender': 'Female', 'Occupation':  
'Student'}
```

Dictionary Data Add/ Remove

#Add Item

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}
thisdict["color"] = "red"
print(thisdict)
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
```



#Remove Item

```
thisdict.pop("model")
print(thisdict)
```

Output:

```
{'brand': 'Ford', 'year': 1964, 'color': 'red'}
```

For more details, read here

https://www.w3schools.com/python/python_dictionaries_remove.asp

Looping Dictionary

```
person_dict = { "Tom": "Male", "Lisa": "Female"}  
for person in person_dict: #Loop gets keys only  
    print(person)
```

Output:

Tom
Lisa



```
#Use .items method to get key and value  
for name, gender in person_dict.items():  
    print(name, gender, sep=": ")
```

Output:

Tom: Male
Lisa: Female



Nested Dictionary

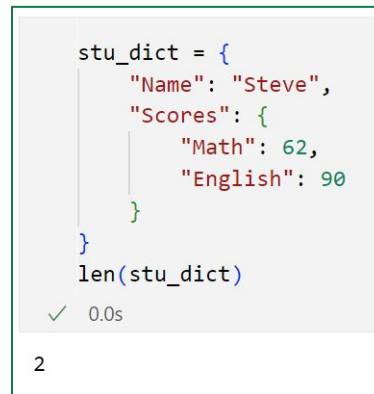
Dictionary can be **nested**, where a value of a key-value pair contains another dictionary inside. Nesting is **not exclusive** to dictionary, it can be done on lists, tuples, sets as well.

#Nested Dictionary in a Dictionary

```
student_dictionary = {  
    "Name": "Steve",  
    "Scores": {  
        "Math": 62,  
        "English": 90  
    }  
}  
student_dictionary["Scores"]["Math"]
```

OUTPUT:

62



```
stu_dict = {  
    "Name": "Steve",  
    "Scores": {  
        "Math": 62,  
        "English": 90  
    }  
}  
len(stu_dict)
```

✓ 0.0s

2

#Nested List (Inside 1 list and 1 tuple)

```
my_list = [1, 2, [3, 4], (5, 6)]
```

QUIZ Time

19

Time Duration: 5 Mins

Instructions: You will work with a nested dictionary representing a customer's shopping cart. Follow the steps below to complete the quiz.

1. **Print the price of "Pan"**
 - Using the **shopping** dictionary provided, write a line of code to extract and print the price of the **"Pan"**.
2. **Calculate and Add a New Key(Total)**
 - **Goal:** Calculate the total cost of all the products purchased by the customer.
 - **Hint:** You'll need to use a **for** loop to go through each item and sum up its price.
 - Once you have the total, add a new key-value pair to the **Customer** dictionary. The new key should be **"Total"**, and its value should be the calculated sum.
3. **Final Printout**
 - After completing the steps above, print the updated **shopping** dictionary to see your changes.

```
shopping = {  
    "Customer": {  
        "Name": "Samson",  
        "Products": {  
            "Pan": 150,  
            "Knife": 80  
        }  
    }  
}
```

QUIZ Time

SOLUTION

```
shopping = {  
    "Customer": {  
        "Name": "Samson",  
        "Products": {  
            "Pan": 150,  
            "Knife": 80  
        }  
    }  
}  
  
#1. print the price of the "Pan".  
print(shopping["Customer"]["Products"]["Pan"])
```

OUTPUT: 150

QUIZ Time

21

SOLUTION

```
shopping = {  
    "Customer": {  
        "Name": "Samson",  
        "Products": {  
            "Pan": 150,  
            "Knife": 80  
        }  
    }  
}
```

#2. Calculate and Add a New Key(Total)

```
total = 0  
for price in shopping["Customer"]["Products"].values():  
    total = total + price  
shopping["Customer"]["Total"] = total  
print(total)
```

OUTPUT: 230

QUIZ Time

22

SOLUTION

#3. Final Printout

```
print(shopping)
```

OUTPUT:

```
{'Customer': {'Name': 'Samson', 'Products': {'Pan': 150,  
'Knife': 80}, 'Total': 230}}
```

About Sets in Python

What is Set?

- A set in Python is an **unordered collection** of unique, **immutable** items.
- It's designed for operations where you care about the presence of an item, but **not its position or count**.

Why Set?

- **Removing Duplicates:** Sets are the easiest way to remove duplicate items from a list or other collection. Simply convert the collection to a set, and Python will handle the rest.
- **Mathematical Operations:** Sets are designed for operations like finding the **intersection**, **union**, or **difference** between two collections.

How to use them?

You create a set by listing items inside curly braces {}

```
unique_fruits = {"apple", "banana", "apple"} # Python automatically removes the duplicate apple.
```

Set declaration

Set can be declared in two ways:

1. by directly creating one with **curly brackets { }**,
2. by converting from other collections such as list or tuples using **set() function**.

```
x = {1, 2, 5, 6}
```

```
type(x)
```

✓ 0.0s

set

```
lst = [1, 5, 5, 8, 1, 'a', 'b', 'b']
```

```
set_a = set(lst)
```

```
set_a
```

✓ 0.0s

```
{1, 5, 8, 'a', 'b'}
```


Union operation

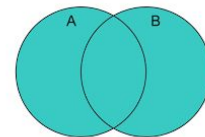
Union combines elements of two sets together.

We use the pipe symbol "`|`" as the union operator.

```
# Union
set_a = {1, 2, 5, 6}
set_b = {5, 1, 9}
set_a | set_b
```

✓ 0.0s

{1, 2, 5, 6, 9}



Intersection operation

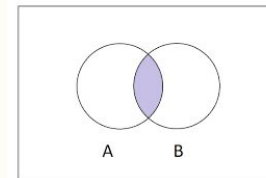
Intersection selects only elements that exist in both sets.

We use the ampersand symbol "`&`" as the intersection operator.

```
# Intersection
set_a = {1, 2, 5, 6}
set_b = {5, 1, 9}
set_a & set_b
```

✓ 0.0s

{1, 5}



Difference operation

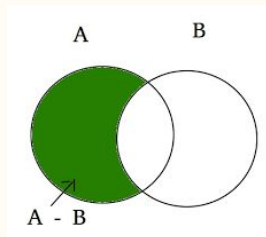
Difference of set A and B results in a set of all elements in set A but not in B.

We use hyphen "`-`" as the difference operator.

```
# Difference
set_a = {1, 2, 5, 6}
set_b = {5, 1, 9}
set_a - set_b
```

✓ 0.0s

{2, 6}



Homework Name: Lab1HW3

1. List Flattening (1 Point)

Instructions:

- You are given a nested list representing a matrix: matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]].
- Write a program using either a for loop or a list comprehension to "flatten" this matrix.
- Desired Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]

2. Student Score Analysis (2 Points)

Instructions:

You are given a list of dictionaries containing student information and scores.

Your program must perform the following three steps:

1. **Add Data:** Add two new student dictionaries to the stu_scores list.
Feel free to use your own names and scores for any subjects you choose.
2. **Calculate Average:** For each student, calculate their average score across all subjects and add it as a new key-value pair to their dictionary.
The key should be **"Average"**.
3. **Find Top Subject:** For each student, find their highest-scoring subject and add it as a new key-value pair to their dictionary. The key should be **"Top"**.

Homework Name: Lab1HW3

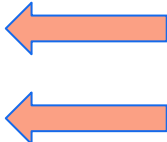
2. Student Score Analysis (2 Points) - CONTINUED

Initial Data:

```
stu_scores = [{  
    "Name": "James",  
    "Scores": {  
        "Math": 22,  
        "Science": 43  
    }  
}]
```

Example Final Output:

```
stu_scores = [{  
    "Name": "James",  
    "Scores": {  
        "Math": 22,  
        "Science": 43  
    },  
    "Average": 32.5,  
    "Top": "Science"  
},  
# ...other students  
]
```



Homework Name: Lab1HW3

3. Character Frequency Counter (2 Points)

Instructions:

- Write a program that asks the user for a word or a short sentence.
- Count how many times each character appears and store the result in a dictionary.
- The program should print this dictionary at the end.

Example:

If the user enters the word hello, the output should be:

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

Hint:

To solve this, you can use a dictionary to keep track of the character counts.

Start with an empty dictionary.

Use a for loop to go through each character in the user's input.

Inside the loop, for each character, check if it already exists as a key in your dictionary.

If the character is already a key, increase its corresponding value (the count) by 1.

If the character is not a key, add it to the dictionary with a starting value of 1.

Homework Instructions

Submission Format:

- Please submit your assignments as either a **.ipynb** (Jupyter Notebook) file or a **.pdf** file.
- If you choose to submit a **.pdf**, make sure all of your code and its output are **clearly visible**.

File Naming Convention:

- You must name your file exactly as shown below:
- YourName_YourStudentID_AssignmentName.FileExtension
- **Example:** Anushka_st126222_Lab3HW.pdf

Late Submissions:

- Please submit all assignments **on time**. Late submissions will be **penalized**.
- Submissions made **within 24 hours after the deadline** will receive **50% penalty**.
- Submissions will **not be accepted** more than **24 hours after the deadline**.

Resource for Further Studies into Python



<https://www.w3schools.com/python/>

Any Questions?

Send Email at: st126222@ait.ac.th